



Week 9 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@November 1, 2021 12:03 PM
🔗 Materials	
# Module #	49
▼ Type	Lecture
# Week #	9

Security Mechanisms

Types of security checks

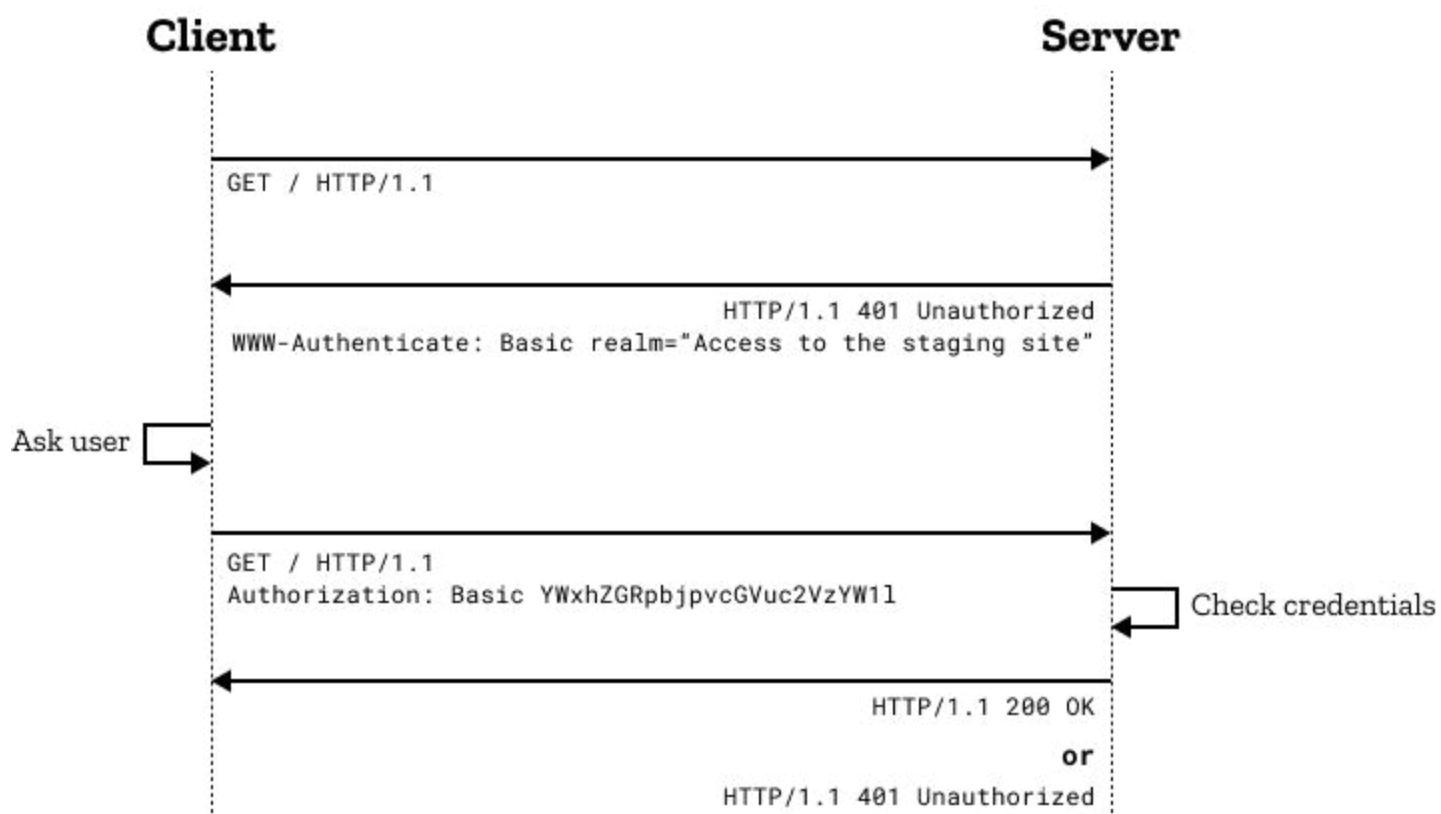
- Obscurity (generally a very bad idea)
 - application listens on non-standard port known only to specific people
- Address →
 - where are you coming from? host based access/deny controls
- Login →
 - username/password provided to each person needing access
- Tokens →
 - access tokens that are difficult/impossible to duplicate
 - can be used for machine-to-machine authentication without passwords

HTTP authentication

Basic HTTP auth →

- Enforced by the server
- Server returns "401/Unauthorized" code to the client
- Contrast with →
 - 404 → not found
 - 403 → forbidden (no option to authenticate)

- Client must respond with access token as an extra "Header" in the next request



Source: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication>

Problems with HTTP Basic Auth

- Username, Password effectively sent as plain text (base64 encoding)
 - Some minimal security if HTTPS is used (wiretap is difficult)
- Password will be seen in cleartext at the server
 - Should not be needed → better mechanisms possible
- No standard process for "logout"

Digest Authentication

- Message digest → cryptographic function
 - eg. MD5, SHA1, SHA256 etc.
- One-way function
 - $f(A) = B$
 - Easy to compute B given A
 - Very difficult (nearly impossible) to compute A given B
- Can define such one-way functions on strings
 - String → Binary number

HTTP Digest Authentication

- Server provides a "nonce" (Number used once) to prevent snooping
- Client must create a secret value including nonce
- Example
 - $HA1 = MD5(\text{username}:\text{realm}:\text{password})$
 - $HA2 = MD5(\text{method}:\text{URI})$
 - $\text{response} = MD5(HA1:\text{nonce}:HA2)$
- Server and client know all the parameters above, so both will compute the same
- Any third party snooping will see only final response
 - cannot extract original values (username, password, nonce, etc)
 - nonce only used once to prevent replay

Client certificates

- Cryptographically secure credentials provided to each client
- Client does not handshake with the server to exchange information, prove knowledge
- Keep cert secure on client end
 - Impossible to reverse and find the key

Form Input

- Username, Password entered into the form
- Transmitted over link to server
 - link must be kept secure (HTTPS)
- GET requests
 - URL encoded data → very insecure, open to spoofing
- POST requests
 - form multipart data → slightly more secure
 - still needs secure link to avoid data leakage

Request level security

- One TCP connection
 - One security check may be sufficient
 - other network level issues to consider for TCP security
- Without connection `keepAlive` →
 - each request needs new TCP connection
 - each request needs new authentication

Cookies

- Server checks some client credentials, then "sets a cookie"
- Header
 - `Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value>; Secure; HttpOnly`
- Client must send back the cookie with each request
- Server maintains "sessions" for clients
 - Remember cookies
 - Can set timeouts
 - Delete cookie records to "logout"
- Client
 - must send cookies with each request

API security

- Cookies etc. requires interactive use (browser)
- Basic auth pop-up window

APIs →

- Typically accessed by the machine client or other applications
- Command-line etc. possible
- Use "token" or "API key" for access
 - subject to same restrictions → HTTPS, not part of the URL, etc